ENSE810 Semester 2, 2023

Individual Project

Student name: Joshua Anto

Student ID: 17988440

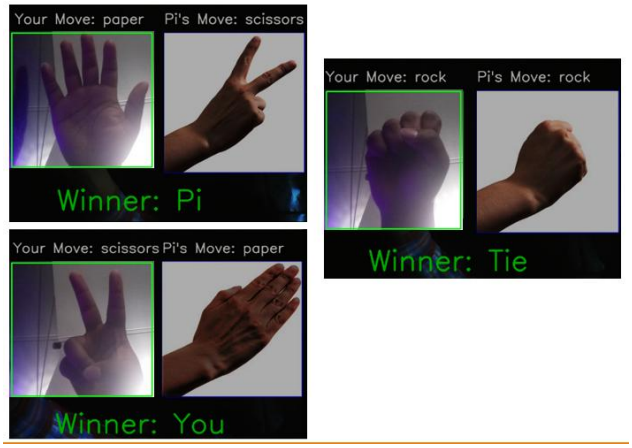
Project title: Rock Paper Scissors with OpenCV and TensorFlow

General instructions:

- Justify your answers.
- Your answers should be based on evidence where appropriate.
    - For example, you can refer to source code files, screenshots, the appendix in this report template or other supplementary files submitted with the source files as evidence. The evidence does not count towards the word limit.
    - Images, illustrations etc. may also support the clarity of your statements.
- The evidence itself is not marked (it just supports your answers here in this document).
- Answers exceeding the word limit will be tolerated but not given more marks than short answers.
- Marks are given for correctness, depth of the discussion, difficulty/technical skills, originality of the project topic and implementation, academic writing.

1. User requirements

The main goal of the software is to let users use their computer's camera and gesture recognition technology to play the popular Rock-Paper-Scissors game. It must provide a user-friendly interface with instructions and details about the game's rules and the moves of the opponent that are easy for users to understand and navigate. Real-time hand movements made by the user should be accurately recognised and responded to by the programme, which should also simulate an opponent's moves using a trained model. The programme should also be responsive, accessible, compatible with popular operating systems, and error-handling friendly.



2. Embedded system:

Due to the integration of hardware and software components, the finished project successfully captures the essence of an embedded system. The hardware platform that serves as the system's basis is the Raspberry Pi at its heart. In addition to the hardware, the project makes use of software technologies like TensorFlow for machine learning model training and OpenCV for efficient image data gathering. Embedded systems stand out for their capacity to coordinate certain hardware and software to carry out targeted operations with accuracy and efficiency.

3. Requirements analysis

Functional requirements:

Accurately identifying and translating user hand movements for "rock," "paper," and "scissors" was a fundamental requirement. Gameplay and user engagement are directly impacted by its functionality. To ensure the game's functioning and competitiveness, the project also required a pre-trained model to simulate an opponent's moves depending on user gestures. It was also crucial to follow the Rock-Paper-Scissors game's regulations. the programme required to decide each round's victor.

Non-functional requirements:

For user happiness, it was essential to guarantee real-time and responsive gesture recognition. Performance standards were established to reduce lag and guarantee fluid gameplay. Additionally, the precision of gesture recognition was not necessary for functionality. For a flawless gaming experience, the software had to accurately recognise the user's motions. To be usable by a variety of users, the software also needed to be user-friendly, with simple instructions and an intuitive design. The architecture of the programme should also be expandable to allow for potential future improvements or advancements in gesture detection, even though this is not immediately necessary.

To ensure that the software met these requirements, I have conducted extensive testing such as functional testing, user testing, performance testing, and accuracy testing.

4. Embedded technologies

I used a variety of technologies that were covered in the lab and lectures extensively to construct this project, and each one of them significantly contributed to its effective completion.

Linux Environment:

On the Raspberry Pi, we used the Linux operating system because of its dependability, stability, and compatibility with a wide range of apps. As a result, we were able to build a solid framework for our embedded system.

Raspberry Pi:

On the Raspberry Pi, we used the Linux operating system because of its dependability, stability, and compatibility with a wide range of apps. As a result, we were able to build a solid framework for our embedded system.

Python:

Python was our primary programming language, facilitating rapid development and integration of libraries like OpenCV and TensorFlow for image processing and machine learning. Its readability and extensive libraries were advantageous in building gesture recognition and game logic. Example of python code.

```
# Function to calculate the winner of the game
def calculate_winner(user_move, Pi_move):
    if user_move == Pi_move:
        return "Tie"

    elif user_move == "rock" and Pi_move == "scissors":
        return "You"

    elif user_move == "rock" and Pi_move == "paper":
        return "Pi"

    elif user_move == "scissors" and Pi_move == "rock":
        return "Pi"

    elif user_move == "scissors" and Pi_move == "paper":
        return "You"

    elif user_move == "paper" and Pi_move == "rock":
        return "You"

    elif user_move == "paper" and Pi_move == "scissors":
        return "Pi"
```

OpenCV:

A reliable and responsive gesture detection system for the Rock-Paper-Scissors game was developed using OpenCV, thanks to its flexibility, large library of computer vision capabilities, and cross-platform compatibility. It facilitated the project's emphasis on delivering an engaging user experience while utilising the power of computer vision by streamlining challenging image processing operations. Below is an code example of how OpenCV was implemented.

```
 cv2.rectangle(frame, (10, 30), (310, 330), (0, 255, 0), 2) #Creates
rectangle box to place hand inside

    k = cv2.waitKey(1)
    if k == ord('r'): #if "k" pressed create rock image directory
            name = 'rock'
            IMG_CLASS_PATH = os.path.join(IMG_SAVE_PATH, name)
            os.mkdir(IMG_CLASS_PATH)
```

```
    if k == ord('p'):
            name = 'paper' # if "k" pressed create paper image directory
            IMG_CLASS_PATH = os.path.join(IMG_SAVE_PATH, name)
            os.mkdir(IMG_CLASS_PATH)

    if k == ord('s'):
            name = 'scissors' # if "k" pressed create scissors image
directory
            IMG_CLASS_PATH = os.path.join(IMG_SAVE_PATH, name)
            os.mkdir(IMG_CLASS_PATH)

    if k == ord('n'):
            name = 'nothing' # if "k" pressed create nothing image directory
            IMG_CLASS_PATH = os.path.join(IMG_SAVE_PATH, name)
            os.mkdir(IMG_CLASS_PATH)
```

Multithreading:

In my project there was no implementation of multithreading . the project captures webcam input, recognize hand gestures and determines the winner each round. The code can be improved upon by implementing multithreading to improve performance and responsiveness.

Version Management:

This project did not necessarily require the use of git for version control as it was a individual project and did not involve multiple parties editing the software. However, in the future if the project wishes to be built upon by multiple teams, then the use of git would be predominantly important.

5. Testing

A thorough testing method was diligently used throughout the creation of our project to make sure that its functionality, performance, and usability complied with both functional and nonfunctional requirements.

One of the main components of our quality control procedure was functional testing. I carefully examined the implementation of game logic the hand gesture recognition and evaluated the success of the opponent simulation. The primary goal of this testing step was to ensure that the fundamental gameplay mechanics complied with the rules of the traditional Rock-Paper-Scissors game.

Performance testing was needed if we were to create a responsive and lag-free user experience. I carefully analysed any visible lag as well as the responsiveness to user inputs. The software needed to be optimised for seamless real-time interaction, therefore this stage was essential. Incorporating multithreading would have been useful to improve performance and responsiveness, however, could increase complexity of the software.

A crucial part of the development approach was user testing. My peers used the software to test its usability, offering priceless insight on the user interface's usability, intuitiveness, and overall user experience. Their advice was crucial in improving the application's user-centric features.

Gesture recognition was thoroughly examined throughout accuracy testing. To assure accuracy and consistency in recognising user inputs, a variety of hand gestures were evaluated. This stage was crucial in perfecting the system's ability to recognise a variety of movements.

Iterative development, user feedback, and continuous testing were crucial to not only satisfying but exceeding both functional and nonfunctional criteria.

Optional: Appendix or references

(unmarked)